

Python para Entrevistas Técnicas

Fase 1 · Sesión 1 · Variables y Tipos

Desde C# · Estilo Kumon

 Concepto Python completo con puente C#	 Calentamiento Sintaxis repetida sin pensar	 LeetCode Problema + variantes	 Entrevista Preguntas resueltas
--	--	---	--

Bloque 1 — Concepto

¿Qué es Python?

Python es un lenguaje interpretado, de tipado dinámico y de propósito general. Eso significa tres cosas concretas para ti que vienes de C#:

Las 3 diferencias más importantes vs C#

1. Sin declarar tipos → Python los infiere en tiempo de ejecución
2. Sin punto y coma → el salto de línea termina la instrucción
3. Indentación = estructura → lo que en C# son { }, en Python es el espacio

Variables y tipos primitivos

En C# escribes el tipo antes de la variable. En Python simplemente asignas:

C# (lo que ya sabes)	Python (lo nuevo)
<code>int edad = 28;</code>	<code>edad = 28</code>
<code>double sueldo = 35000.50;</code>	<code>sueldo = 35000.50</code>
<code>string nombre = "Ana";</code>	<code>nombre = 'Ana' # o "Ana"</code>
<code>bool activo = true;</code>	<code>activo = True # mayúscula</code>
<code>object x = null;</code>	<code>x = None # null en Python</code>
<code>// comentario de línea</code>	<code># comentario de línea</code>

Tipos numéricos

Python tiene int y float. No existe double, long ni decimal como tipos separados:

```

x = 5          # int
y = 3.14       # float
z = 2 ** 10    # potencia → 1024 (en C# era Math.Pow(2, 10))
a = 10 // 3    # división entera → 3 (en C# era 10 / 3 con ints)
b = 10 % 3     # módulo → 1 (igual que C#)

type(x)        # → <class 'int'>
type(y)        # → <class 'float'>

```

Strings y f-strings

Los strings en Python van con comillas simples o dobles — ambas son válidas. Para interpolar variables usas f-strings, que son el equivalente al \$ de C#:

C# (lo que ya sabes)	Python (lo nuevo)
\$"Hola {nombre}"	f'Hola {nombre}'
\$"Tiene {edad + 1} años"	f'Tiene {edad + 1} años'
int.Parse("42")	int('42')
Convert.ToString(42)	str(42)
"hola".ToUpper()	"hola".upper()
"hola".Length	len("hola")

```

nombre = 'Ana'
edad = 28
sueldo = 35000.50

print(f'{nombre} tiene {edad} años y gana ${sueldo:.2f}')
# → Ana tiene 28 años y gana $35000.50

# Conversiones de tipo
numero = int('42')      # string → int
texto = str(42)         # int → string
decimal = float('3.14') # string → float

```

Operadores de comparación y booleanos

Idénticos a C# excepto los valores booleanos que van en mayúscula y los operadores lógicos que son palabras en español/inglés:

C# (lo que ya sabes)	Python (lo nuevo)
x == y, x != y	x == y, x != y (igual)
x > y, x < y	x > y, x < y (igual)
&& !	and or not

true false	True False
null	None
x == null	x is None (forma correcta)

```

x = 10
y = 20

print(x > 5 and y < 30) # True
print(x == 10 or y == 10) # True
print(not True) # False

nada = None
print(nada is None) # True ← usa 'is', no '=='

```

Condicionales

La estructura if/elif/else reemplaza los { } con dos puntos y sangría:

C# (lo que ya sabes)	Python (lo nuevo)
if (cond) { }	if cond:
else if (cond) { }	elif cond:
else { }	else:
cond ? a : b	a if cond else b

```

edad = 20

if edad >= 18:
    print('Mayor de edad')
elif edad >= 13:
    print('Adolescente')
else:
    print('Menor de edad')

# Ternario (equivale a cond ? a : b en C#)
mensaje = 'adulto' if edad >= 18 else 'menor'

```

Loops

Python tiene for y while. El for es mucho más expresivo que el de C#:

C# (lo que ya sabes)	Python (lo nuevo)
for (int i = 0; i < 5; i++)	for i in range(5):
for (int i = 2; i < 10; i += 2)	for i in range(2, 10, 2):
foreach (var x in lista)	for x in lista:

while (cond) { }	while cond:
break / continue	break / continue (igual)

```
# range(5) → 0, 1, 2, 3, 4
for i in range(5):
    print(i)

# range(inicio, fin, paso)
for i in range(0, 10, 2):    # 0, 2, 4, 6, 8
    print(i)

# while
contador = 0
while contador < 3:
    print(contador)
    contador += 1    # no existe ++ en Python
```

Puntos críticos para no equivocarse

- True y False van con mayúscula — si escribes 'true' Python lo lee como variable no definida
- No existe ++ ni -- en Python → usa += 1 y -= 1
- La indentación es de 4 espacios (no tabs, no 2 espacios)
- None es el equivalente de null — compáralo siempre con 'is None', no con '== None'
- No hay llaves { } — los : al final de if/for/while abren el bloque

Bloque 2 — Calentamiento (sintaxis repetida)

Escribe el código a mano. El objetivo es que la sintaxis salga sin pensar. No hay truco — solo repetición.

Ejercicio 1 — Declara variables (sin ver el concepto)

Escribe en Python el equivalente exacto de este bloque de C#:

```
// C# – convierte todo a Python
int puntos = 150;
string usuario = "jugador1";
double promedio = 8.75;
bool ganador = false;
object dato = null;
```

Tu código Python:

¿Cuál es el tipo de cada variable? Escribe `type(variable)` para cada una:

Tu respuesta:

Ejercicio 2 — f-strings

Completa las líneas para que impriman lo que se indica:

```
nombre = 'Carlos'
edad = 32
ciudad = 'CDMX'

# Debe imprimir: Carlos tiene 32 años
print(_____)

# Debe imprimir: Carlos vive en CDMX y el año que viene tendrá 33
print(_____)

# Convierte el string '99' a entero y súmalo 1
resultado = _____
print(resultado) # debe imprimir 100
```

Escribe las tres líneas completadas:

Ejercicio 3 — Operadores y comparaciones

Escribe el resultado de cada expresión sin ejecutar el código — solo razonando:

```
a = 15
b = 4

a // b          # → ____
a % b           # → ____
2 ** 8          # → ____
a > 10 and b < 10  # → ____
a == 15 or b == 5  # → ____
```

```
not (a == b)      # → ____  
a is None        # → ____
```

Ahora reescribe la última línea para que devuelva True:

Tu respuesta:

Ejercicio 4 — Condicionales de memoria

Escribe una función que reciba una nota (0-100) y devuelva:
'A' si nota >= 90, 'B' si >= 80, 'C' si >= 70, 'F' si menor a 70

Escribe la función completa:

Ejercicio 5 — Loops de memoria

Escribe los loops que producen estas salidas:

a) Imprime los números del 1 al 10:

Tu respuesta:

b) Imprime los múltiplos de 3 entre 1 y 30:

Tu respuesta:

c) Cuenta regresiva: 10, 9, 8 ... 1, ¡despegue! (usa while):

Tu respuesta:

Bloque 3 — LeetCode

Problema base: LC #9 — Palindrome Number

Dado un entero x, devuelve True si x es palíndromo, False si no.
Un número es palíndromo si se lee igual de izquierda a derecha y de derecha a izquierda.

Ejemplos:

x = 121 → True (121 al revés sigue siendo 121)
x = -121 → False (de derecha a izquierda es 121-)
x = 10 → False (01 no es igual a 10)

Restricción importante: ¿puedes resolverlo sin convertir x a string?

Paso 1 — Explica en español qué tiene que hacer tu código

Antes de escribir código, escribe el algoritmo en tus palabras:

Tu respuesta:

Paso 2 — Solución con string (la más simple)

Esta versión convierte el número a string y usa slicing para invertirlo. Completa los huecos:

```
def isPalindrome(x: int) -> bool:
    # Los negativos nunca son palíndromos
    if x < ____:
        return ____

    # Convierte a string
    s = ____ (x)

    # Compara con el string invertido
```

```
# Pista: s[::-1] invierte un string en Python
return s == ____
```

Paso 3 — Escribe la solución completa de memoria

Solución completa (sin ver el paso 2):

Variantes del problema

Ejercicio V1 — Palíndromo con string limpio

Modifica la solución para ignorar espacios y mayúsculas. Por ejemplo:

'A man a plan a canal Panama' → True

'Was it a car or a cat I saw' → True

Pista: usa s.lower() y s.replace(' ', '')

```
def isPalindromeStr(s: str) -> bool:
    limpio = s.lower().replace(' ', '')
    return ____
```

Escribe la línea que falta y explica qué hace [::-1]:

Ejercicio V2 — Palíndromo sin convertir a string

Resuelve el problema original usando solo matemáticas (sin str()).

Pista: puedes invertir el número dígito a dígito usando % 10 y // 10

```
def isPalindromeInt(x: int) -> bool:
    if x < 0 or (x % 10 == 0 and x != 0):
        return False

    invertido = 0
    while x > invertido:
        invertido = invertido * 10 + x % ____
        x //= ____
```

```
return x == invertido or x == invertido // ____
```

Llena los huecos y traza la ejecución con $x = 121$:

Ejercicio V3 — ¿Cuántos palíndromos hay en una lista?

Dado `nums = [121, -121, 0, 11, 10, 1331]`, devuelve cuántos son palíndromos.
Usa tu función `isPalindrome` del paso 3.

Escribe la función de memoria:

¿Qué resultado da con el ejemplo? → ____

Bloque 4 — Preguntas de entrevista

Estas son preguntas reales de entrevistas técnicas. Lee la respuesta modelo, luego tapa la respuesta y practica decirla en voz alta.

Pregunta 1

¿Cuál es la diferencia entre tipado dinámico y tipado estático?

Respuesta modelo:

En tipado estático, como C#, tú declaras el tipo de cada variable antes de usarla y el compilador verifica que nunca lo violes — si declaras un `int` no puedes meterle un `string`. En tipado dinámico, como Python, el tipo lo lleva el valor, no la variable. La misma variable puede apuntar primero a un número y después a un `string`. Eso hace el código más ágil para escribir, pero los errores de tipo solo aparecen en tiempo de ejecución, no antes de correr el programa.

Tip de entrevista: si el reclutador pregunta 'y eso no es un problema?', di que Python tiene `type hints` (edad: `int = 28`) que permiten agregar tipos opcionales para tener las ventajas de ambos mundos.

Pregunta 2

¿Por qué en Python se compara `None` con `'is'` en lugar de `'=='`?

Respuesta modelo:

El operador `==` en Python compara valores — verifica si dos cosas son iguales en contenido. El operador `'is'` compara identidad — verifica si dos variables apuntan exactamente al mismo objeto en memoria. `None` es un singleton en Python: solo existe una instancia de `None` en todo el programa. Por eso la convención es usar `'is None'` — no comparas si algo vale `None`, sino si es literalmente el objeto `None`. Usar `==` técnicamente funciona casi siempre, pero es una mala práctica porque una clase podría redefinir `==` para que `None ==` objeto devuelva `True`.

Tip de entrevista: esto demuestra que entiendes la diferencia entre identidad y igualdad, que es una pregunta clásica en entrevistas de Python senior.

Pregunta 3

¿Cuál es la complejidad de checar si un número es palíndromo?

Respuesta modelo:

Depende del enfoque. Con la solución de string: convertir el número cuesta $O(d)$ donde d es el número de dígitos, invertir el string también $O(d)$, y comparar $O(d)$. Total $O(d)$. Como $d = \log_{10}(n)$, la complejidad real en función del número es $O(\log n)$. Con la solución matemática es igual — $O(\log n)$ — pero sin usar memoria extra para el string. En la práctica para entrevistas se suele decir $O(\log n)$ o simplemente $O(n)$ donde n es el número de dígitos.

Tip de entrevista: el reclutador valora que menciones la complejidad espacial también. La solución con string usa $O(d)$ de memoria extra; la matemática usa $O(1)$.

Pregunta 4

¿Qué es el slicing en Python y cuándo lo usarías?

Respuesta modelo:

El slicing es una sintaxis de Python para extraer porciones de una secuencia — lista, string, tuple — usando la forma `[inicio:fin:paso]`. El índice `fin` es exclusivo, o sea `s[1:4]` te da los elementos en posiciones 1, 2 y 3. El paso es opcional: `s[::2]` te da uno de cada dos, y el famoso `s[::-1]` invierte la secuencia porque va del final al inicio de uno en uno hacia atrás. Lo usaría para invertir strings o listas, para obtener la primera o segunda mitad de un array, o para iterar en reversa sin modificar el original.

Tip de entrevista: menciona el caso `s[::-1]` explícitamente — es un patrón que aparece muchísimo en LeetCode y los reclutadores lo conocen bien.

Resultado final de la sesión

Concepto	Lo domino / Más o menos / Repaso
Declarar variables sin tipo	☐ ☐ ☐
True / False / None con mayúscula	☐ ☐ ☐
Operadores // ** % and or not	☐ ☐ ☐
f-strings con expresiones	☐ ☐ ☐

Condicionales if / elif / else	☐ ☐ ☐
Loops for con range()	☐ ☐ ☐
s[::-1] para invertir un string	☐ ☐ ☐
Comparar con is None	☐ ☐ ☐
Función isPalindrome completa de memoria	☐ ☐ ☐

¿Resolví el LeetCode? ☐ Solo ☐ Con pista ☐ Vi solución	¿Lo entiendo al 100%? ☐ Sí ☐ Más o menos ☐ No	Revisión: ☐ 3 días ☐ 7 días ☐ 30 días
---	--	--

Siguiente sesión → Sesión 2: Listas y slices